

Part-D2

Heaps

الفكرة اتصال root
بأكبر من اولاده أو أصغر من اولاده حسب نوعها.

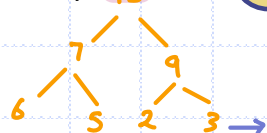
1- (Insert) : يكون (left to right) (of the level)

Priority (min / max)

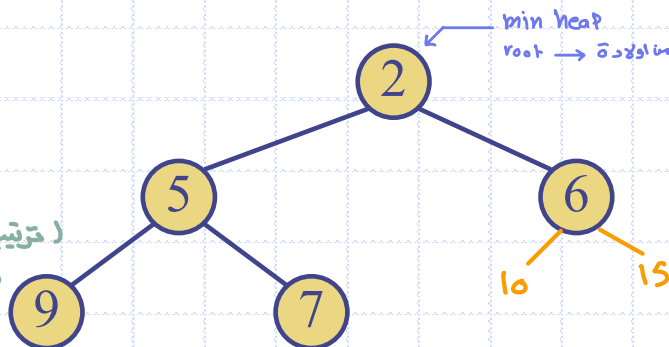
(ترتيب ال node أصغر من أخفاله)

(ترتيب ال node أكبر من أخفاله)

max ← 15



آخر (element)
أخفها



min heap
root → أصغر من اولاده

وفية (max heap) عكس ال min
أكبر من اولاده root →

Recall Priority Queue ADT (§ 7.1.3)

- ◆ A priority queue stores a collection of entries
- ◆ Each **entry** is a pair (key, value)
priority ←
- ◆ Main methods of the Priority Queue ADT
 - **insert(k, x)**
inserts an entry with key k and value x
 - **removeMin()**
removes and returns the entry with smallest key

- ◆ Additional methods
 - **min()** → بترجع أصغر ما متشيل
returns, but does not remove, an entry with smallest key
 - **size(), isEmpty()**
عدد ال elements بترجع ال true or false
- ◆ Applications:
 - Standby flyers
 - Auctions
 - Stock market

1 2 3 4
highest Priority ← → lowest Priority

Recall Priority Queue Sorting (§ 7.1.4)



- ◆ We can use a priority queue to sort a set of comparable elements
 - Insert the elements with a series of **insert** operations
 - Remove the elements in sorted order with a series of **removeMin** operations
- ◆ The running time depends on the priority queue implementation:
 - Unsorted sequence gives selection-sort: $O(n^2)$ time
 - Sorted sequence gives insertion-sort: $O(n^2)$ time
- ◆ *Can we do better?*

Algorithm *PQ-Sort*(S, C)

Input sequence S , comparator C
for the elements of S

Output sequence S sorted in
increasing order according to C

$P \leftarrow$ priority queue with
comparator C

```
while  $\neg S.isEmpty()$   
     $e \leftarrow S.remove(S.first())$   
     $P.insertItem(e, e)$ 
```

```
while  $\neg P.isEmpty()$   
     $e \leftarrow P.removeMin()$   
     $S.insertLast(e)$ 
```

→ the last node of a heap is the rightmost node of maximum depth.

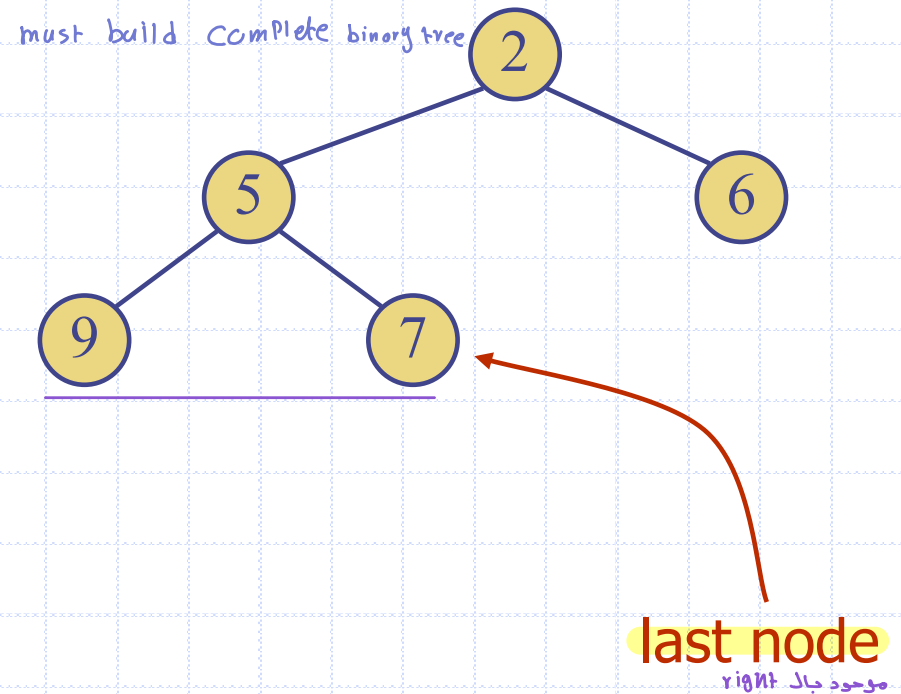
Heaps (§7.3)

First element should be removed
+ return
Max: remove first
min: remove first

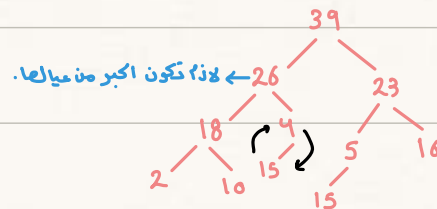
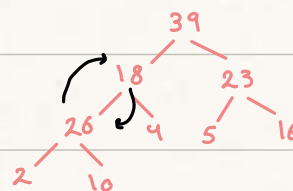
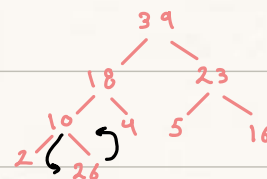
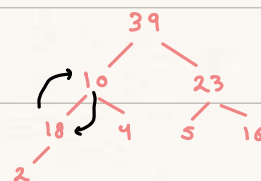
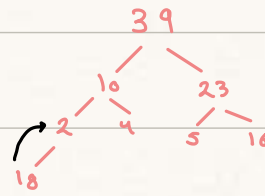
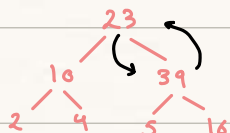
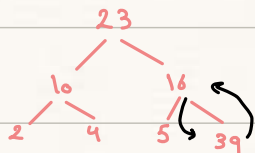
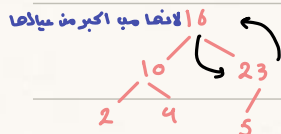
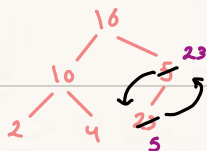
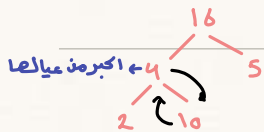
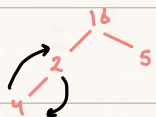
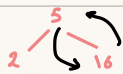
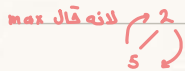
◆ A heap is a binary tree storing keys at its nodes and satisfying the following properties:

- **Heap-Order:** for every internal node v other than the root,
 $key(v) \geq key(parent(v))$
- **Complete Binary Tree:** let h be the height of the heap
 - ◆ for $i = 0, \dots, h - 1$, there are 2^i nodes of depth i
 - ◆ at depth $h - 1$, the internal nodes are to the left of the external nodes

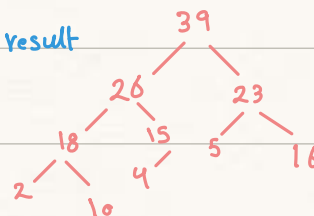
◆ The last node of a heap is the rightmost node of depth h



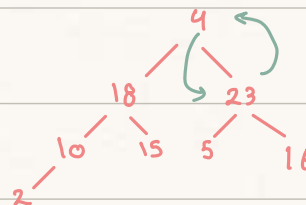
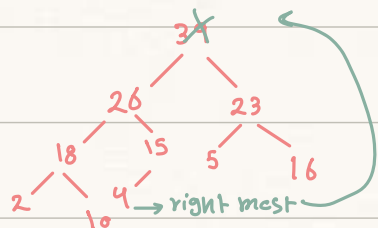
Q1. Illustrate the insertion of the following keys in a heap (**max heap**) (2, 5, 16, 4, 10, 23, 39, 18, 26, 15).



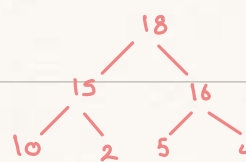
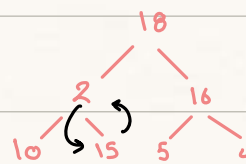
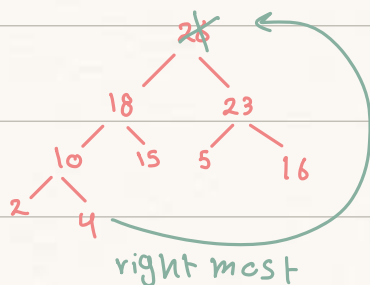
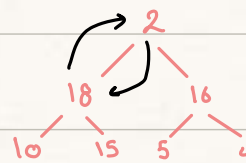
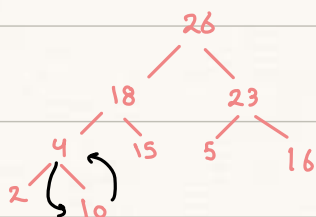
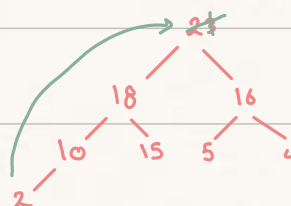
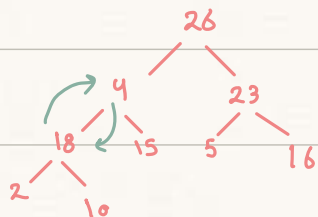
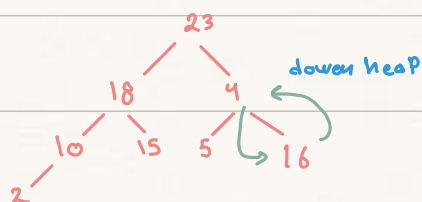
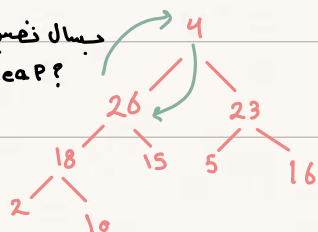
Final result



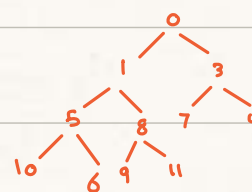
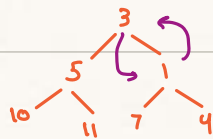
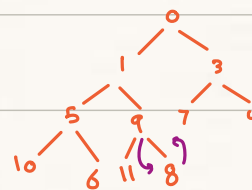
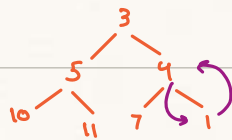
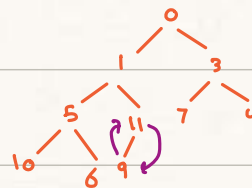
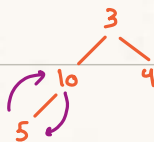
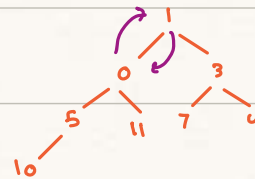
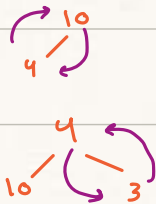
Q2. Illustrate the removal of the following keys from the above heap
39, 26, 23.



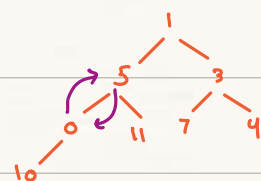
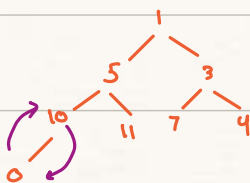
جسٹال نفسی حل می
max heap?
no.



Q3. Illustrate the result of execution of the following integer keys to be inserted in a **priority queue (min heap)**. Each node in the min heap stores an integer. 10, 4, 3, 5, 11, 7, 1, 0, 6, 9, 8



final min heap



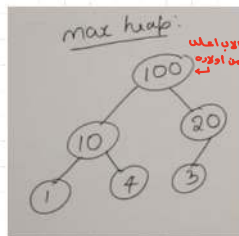
What is a Heap?

A heap is a binary tree that satisfies these special **SHAPE** and **ORDER** properties:

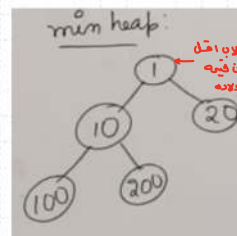
- ▢ **SHAPE** : Its shape must be a **complete binary tree**: a binary tree T with h levels is *complete* if all levels except possibly the last are completely full, and the last level has all its nodes to the left side.
- ▢ **ORDER**: each node in the heap, the value stored in that node is **smaller than or equal to the value in each of its children**.

إذا صاحكت شي ، by default تكون min

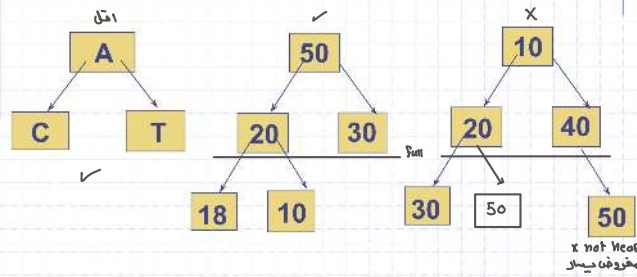
What is a Heap?



إذا node وحده خروطة شي، معناه not heap

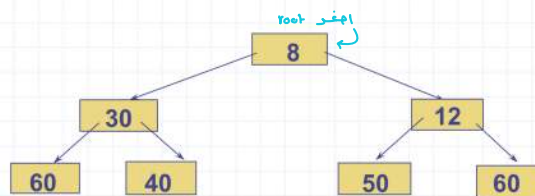


Are these all heaps?



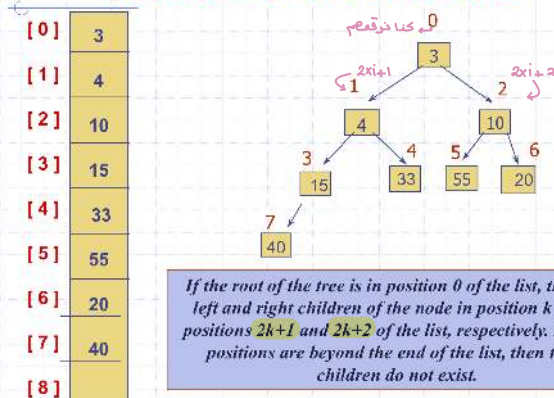
CS 210

Is this a heap? Yes it is



✗ The smallest element in a heap is always found in the root node.

And use the numbers as array indexes to store the tree



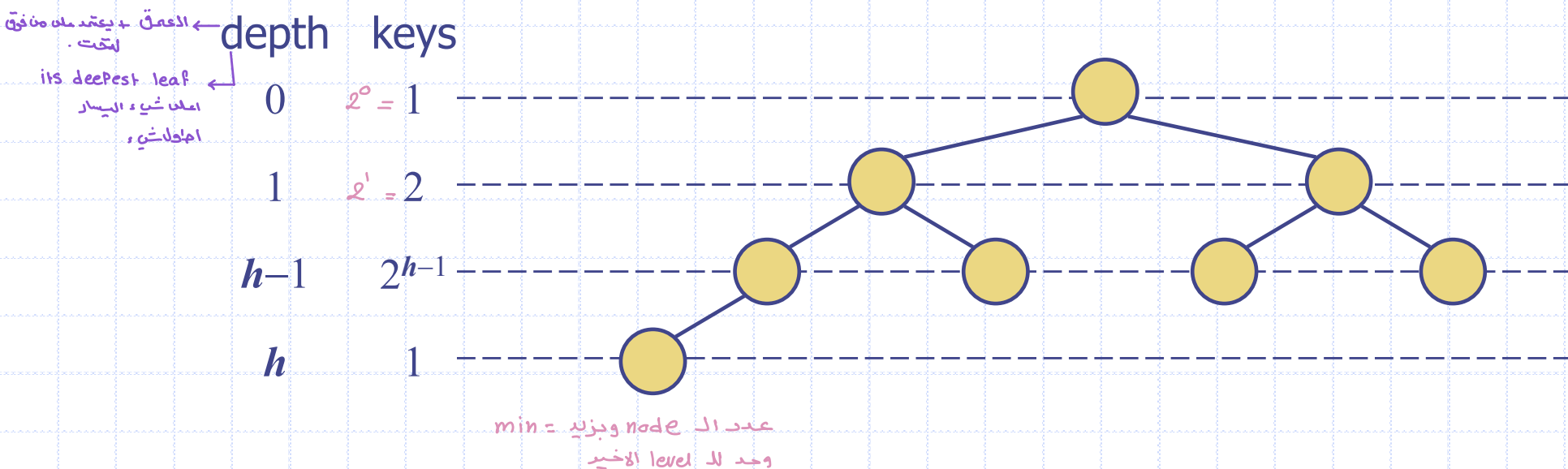
Height of a Heap (§ 7.3.1)



◆ **Theorem:** A heap storing n keys has height $O(\log n)$

Proof: (we apply the complete binary tree property)

- Let h be the height of a heap storing n keys
- Since there are 2^i keys at depth $i = 0, \dots, h-1$ and at least one key at depth h , we have $n \geq 1 + 2 + 4 + \dots + 2^{h-1} + 1$
- Thus, $n \geq 2^h$, i.e., $h \leq \log n$



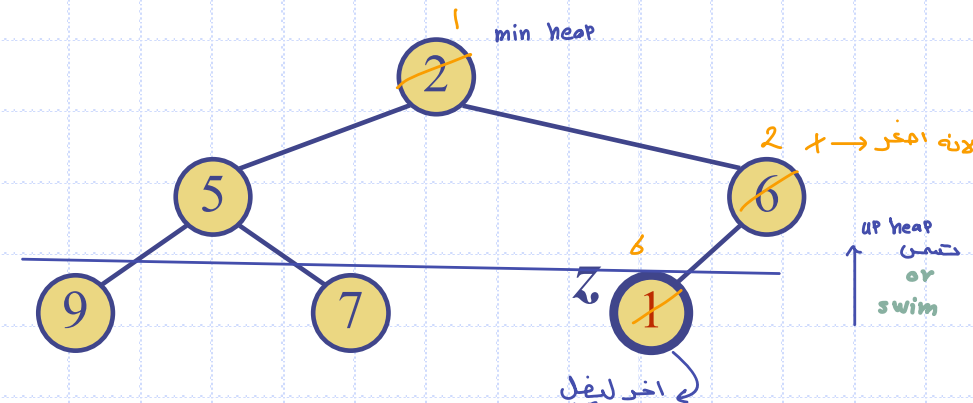
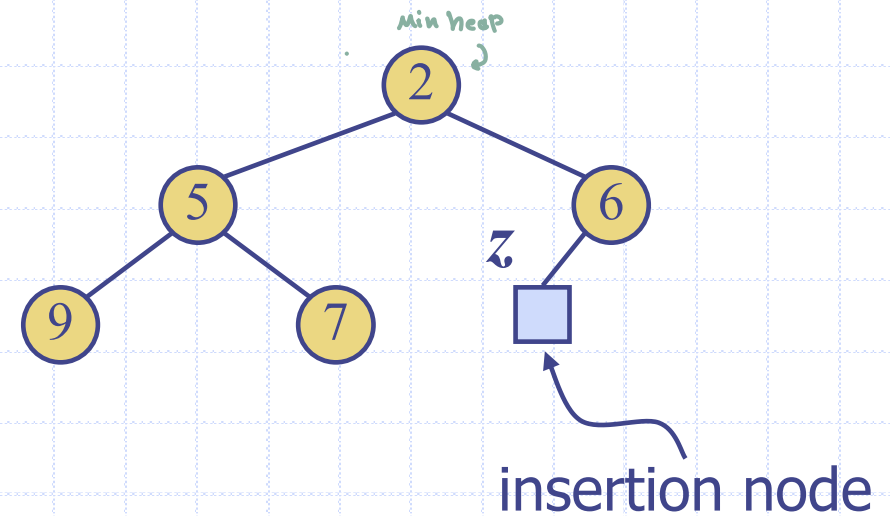
- نفترخا اخر node وظيف
بعدة .



Insertion into a Heap (§ 7.3.3)

دائماً بتجيب جاول Position خافتي
بآخر (level)

- ◆ Method insertItem of the priority queue ADT corresponds to the insertion of a key k to the heap
- ◆ The insertion algorithm consists of three steps
 - Find the insertion node z (the new last node)
 - Store k at z
 - Restore the heap-order property (discussed next)



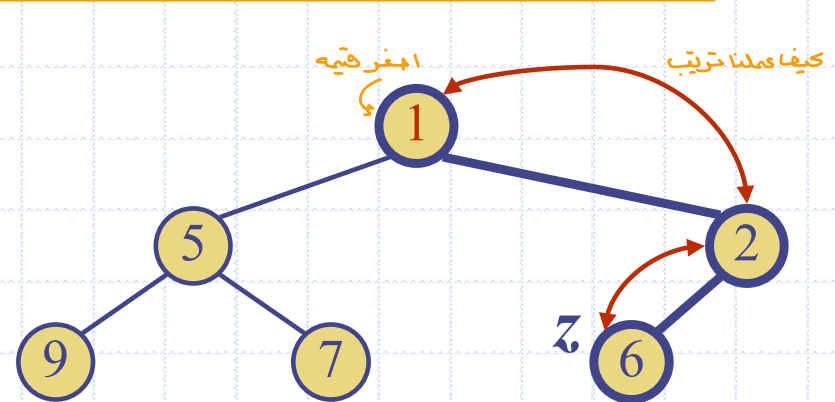
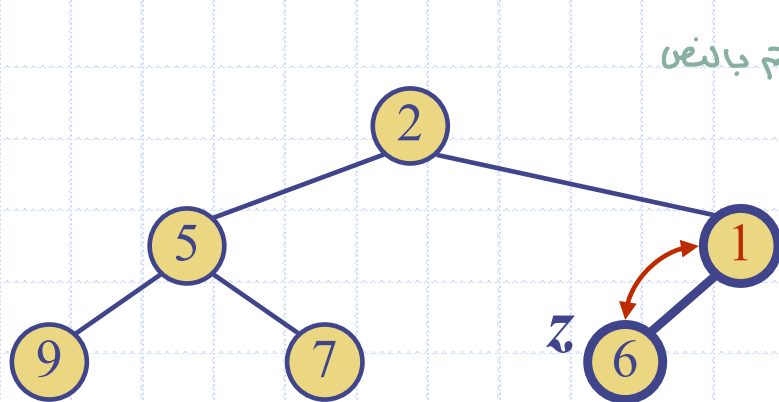
بعدين نرتب على حسب موقعة

شوقيمة بنسبة لـ Parent تبعة هلا بيديل اولاً

Upheap

من تحت لفرق →

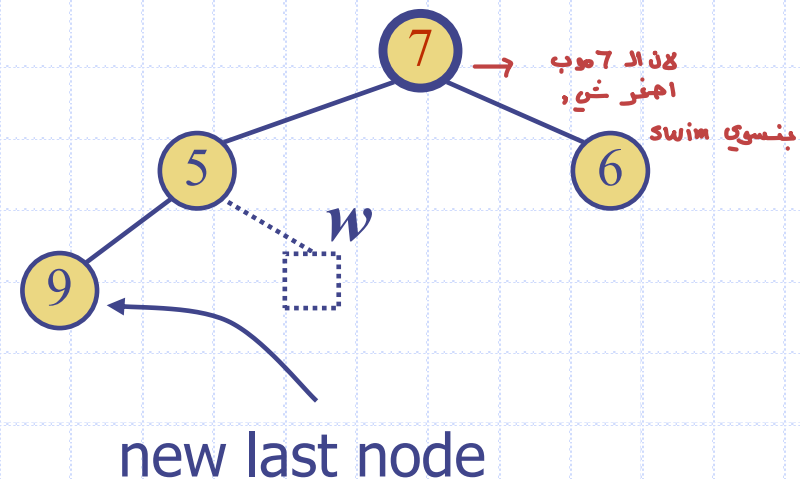
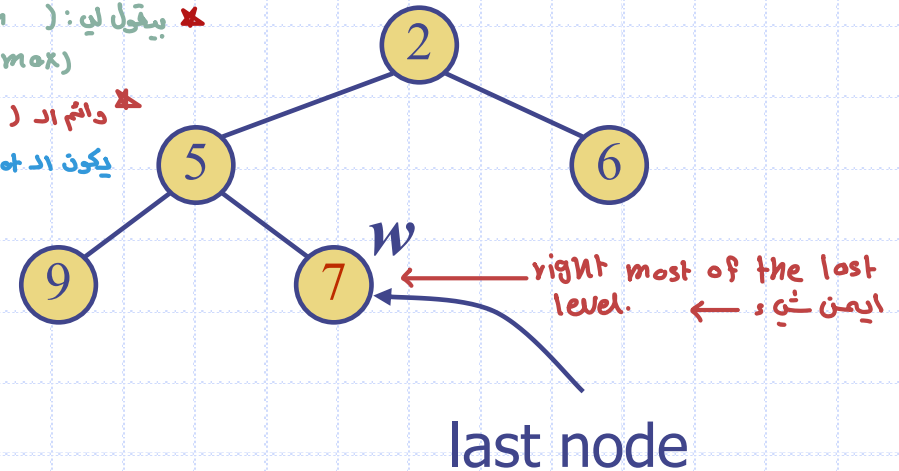
- ◆ After the insertion of a new key k , the heap-order property may be violated
- ◆ Algorithm upheap restores the heap-order property by **swapping** k along an upward path from the insertion node
- ◆ Upheap terminates when the key k reaches the root or a node whose parent has a key smaller than or equal to k
- ◆ Since a heap has height $O(\log n)$, upheap runs in $O(\log n)$ time



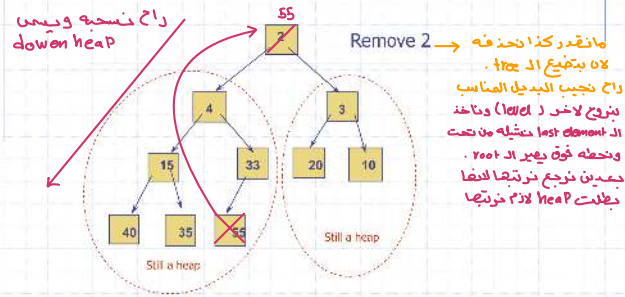
Removal from a Heap (§ 7.3.3)

- ◆ Method `removeMin` of the priority queue ADT corresponds to the removal of the root key from the heap
- ◆ The removal algorithm consists of three steps
 - Replace the root key with the key of the last node w
 - Remove w
 - Restore the heap-order property (discussed next)

✖ يقول لي: `remove min`
or `remove max`
✖ واسم الـ `(delete heap)`
• يكون الـ root

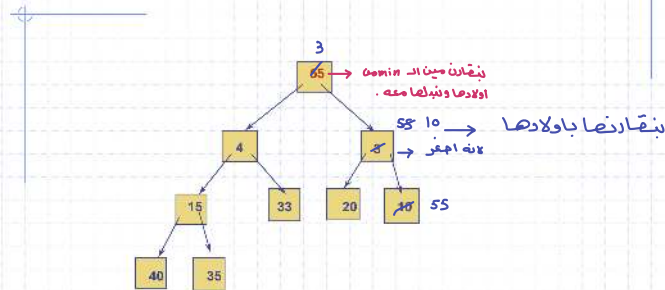


Say we want to remove the element with the smallest value from a heap: removeMin()

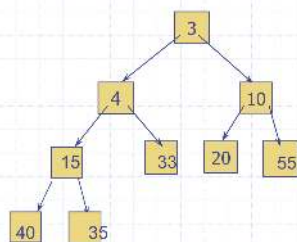


CS 210

Solution: DownHeap.



Finally, a restored heap

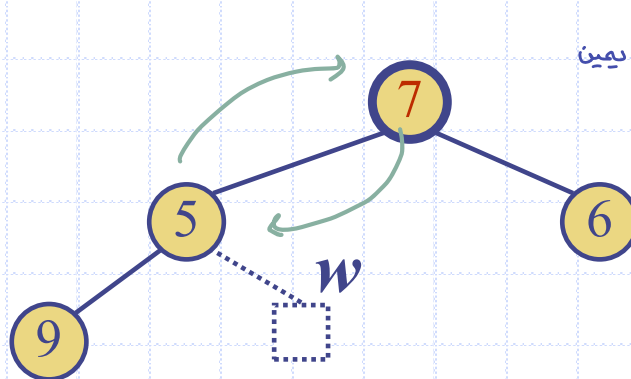


CS 210

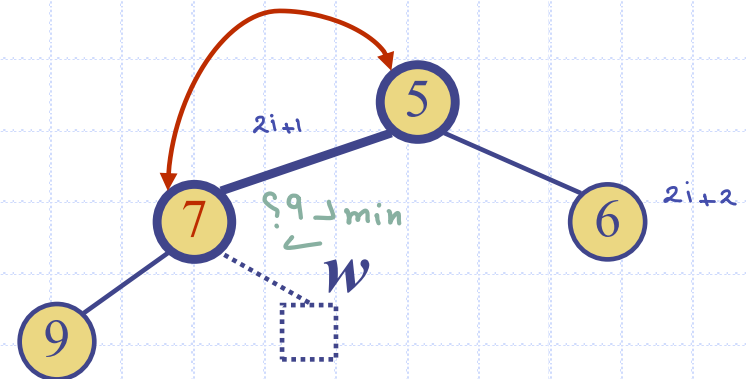
Downheap (remove)

↓
 بنزل بنزل الين ما خوهل لآخر level
 بدون ما نلق min مثاني .

- ◆ After replacing the root key with the key k of the last node, the heap-order property may be violated
- ◆ Algorithm downheap restores the heap-order property by swapping key k along a downward path from the root
- ◆ Upheap terminates when key k reaches a leaf or a node whose children have keys greater than or equal to k
- ◆ Since a heap has height $O(\log n)$, downheap runs in $O(\log n)$ time

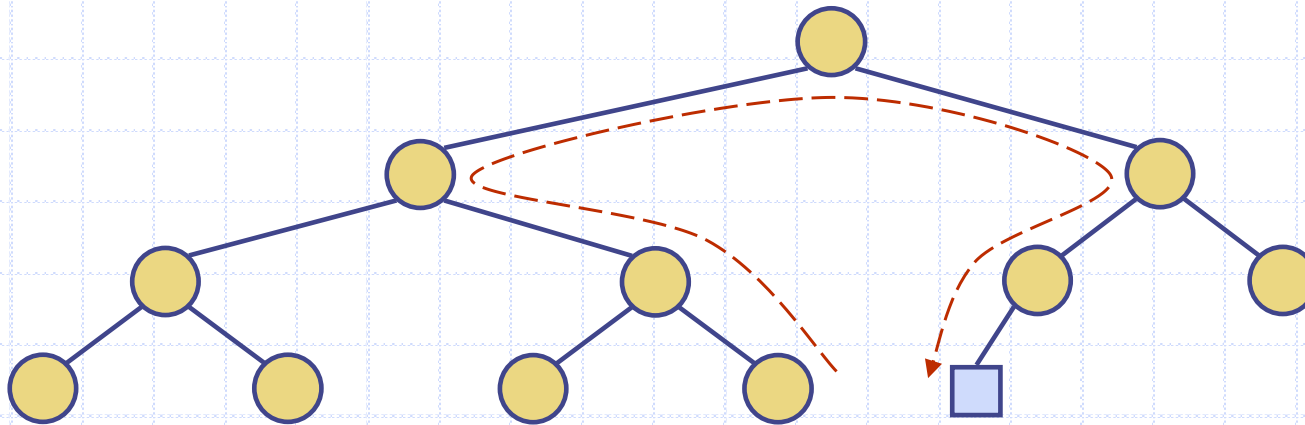


كل مرة بختار يا يسار او يمين
 + شغلي بالنص

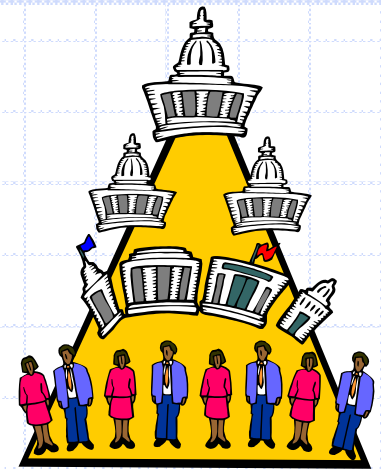


Updating the Last Node

- ◆ The insertion node can be found by traversing a path of $O(\log n)$ nodes
 - Go up until a left child or the root is reached
 - If a left child is reached, go to the right child
 - Go down left until a leaf is reached
- ◆ Similar algorithm for updating the last node after a removal



Heap-Sort (§2.4.4)



◆ Consider a priority queue with n items implemented by means of a heap

- the space used is $O(n)$
- methods **insert** and **removeMin** take $O(\log n)$ time
- methods **size**, **isEmpty**, and **min** take time $O(1)$ time

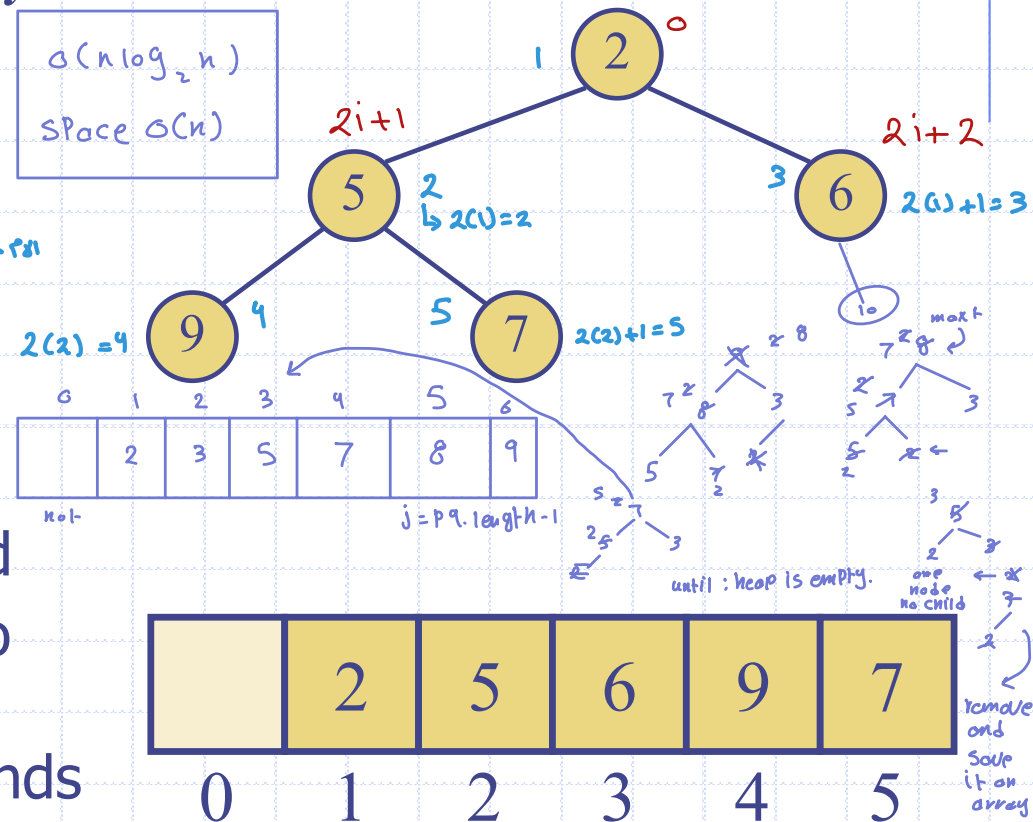
◆ Using a heap-based priority queue, we can sort a sequence of n elements in $O(n \log n)$ ^{run time} _{Space : $O(n)$} time

- ◆ The resulting algorithm is called heap-sort
- ◆ Heap-sort is much faster than quadratic sorting algorithms, such as insertion-sort and selection-sort

Vector-based Heap Implementation (§2.4.3)

- ◆ We can represent a heap with n keys by means of an array of length $n + 1$
- ◆ For the node at rank i
 - the left child is at rank $2i$
 - the right child is at rank $2i + 1$
- ◆ Links between nodes are not explicitly stored
- ◆ The cell of at rank 0 is not used
- ◆ Operation insert corresponds to inserting at rank $n + 1$
- ◆ Operation removeMin corresponds to removing at rank n
- ◆ Yields in-place heap-sort

$O(n \log_2 n)$
Space $O(n)$



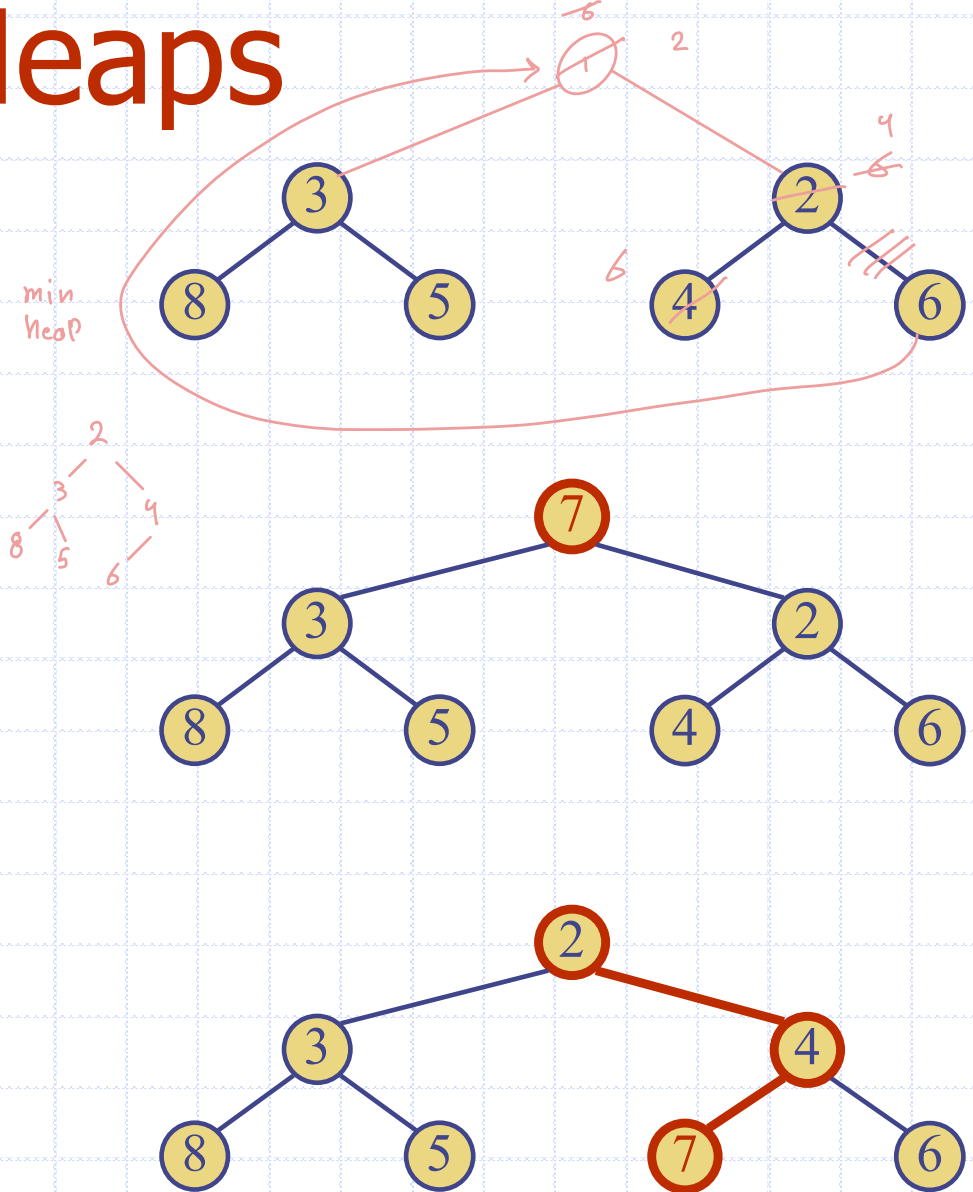
لے دائرہ فانی نیا انجین من

• root ← index (1)

لكن ببال (binary) نیا من الهمفر .

Merging Two Heaps

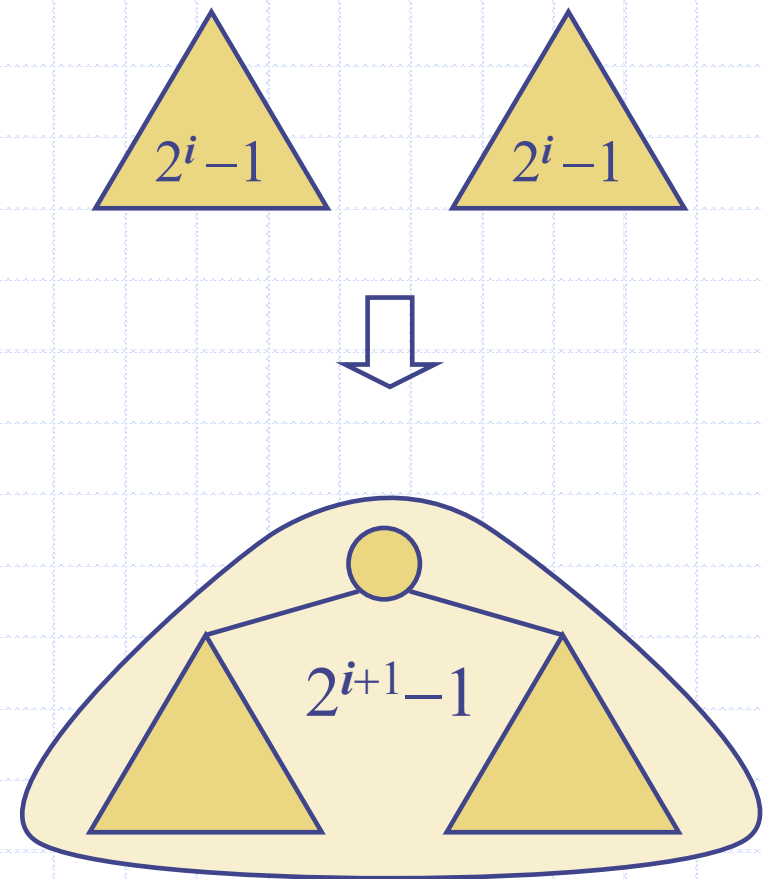
- ◆ We are given two two heaps and a key k
- ◆ We create a new heap with the root node storing k and with the two heaps as subtrees
- ◆ We perform downheap to restore the heap-order property



Bottom-up Heap Construction (§2.4.3)



- ◆ We can construct a heap storing n given keys in using a bottom-up construction with $\log n$ phases
- ◆ In phase i , pairs of heaps with $2^i - 1$ keys are merged into heaps with $2^{i+1} - 1$ keys



Example

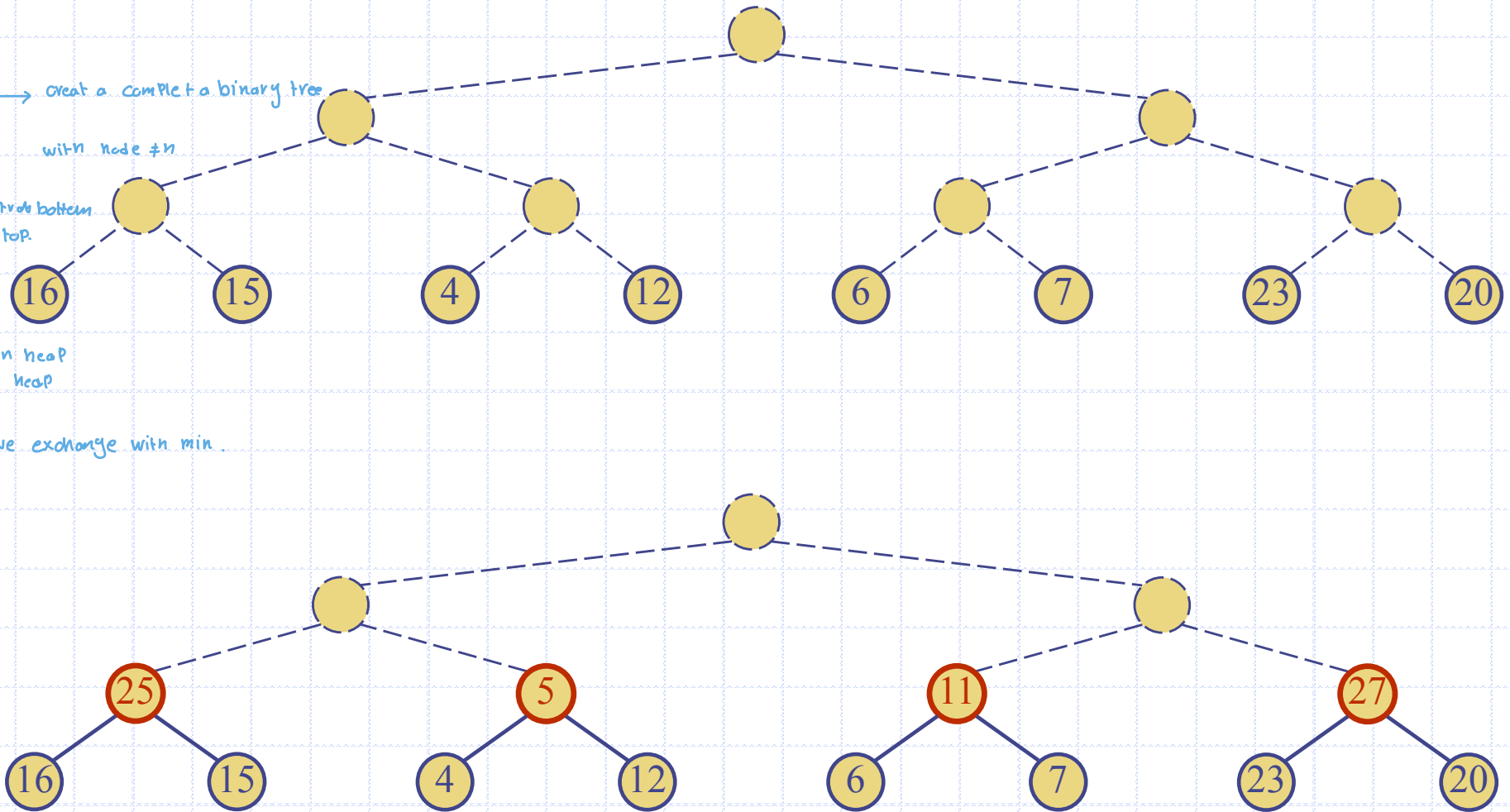
1 → create a complete binary tree

with node #n

2. Sort bottom
to top.

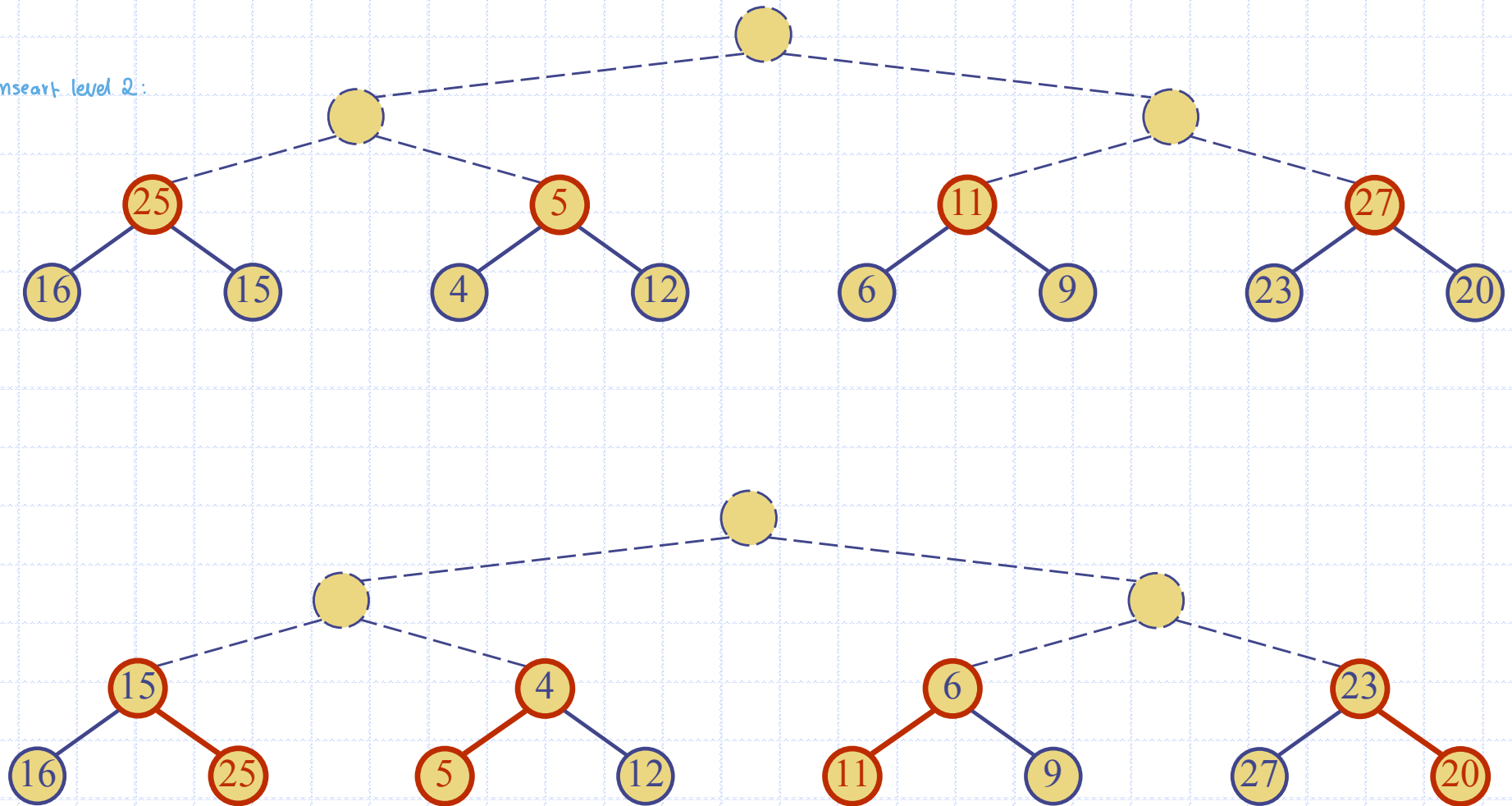
main heap
max heap

* we exchange with min.

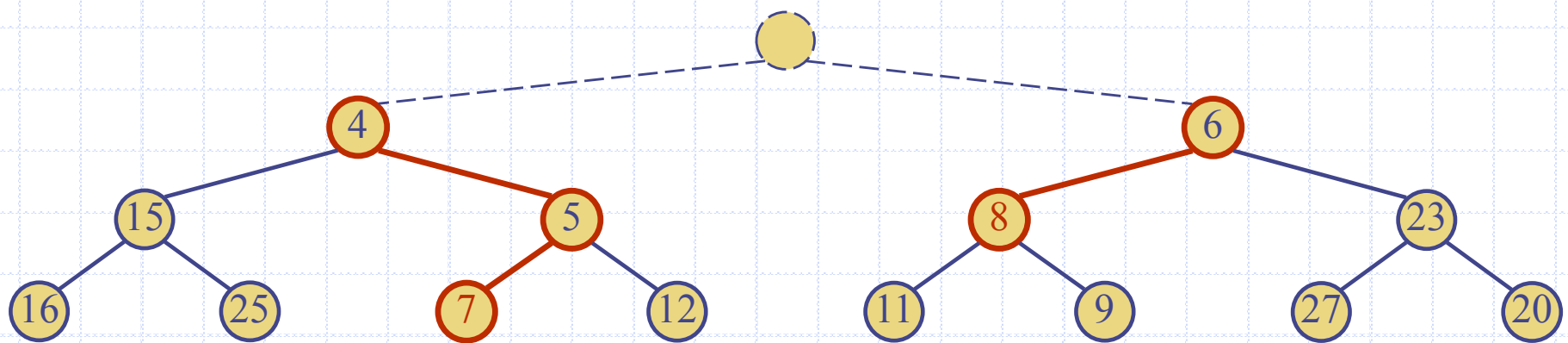
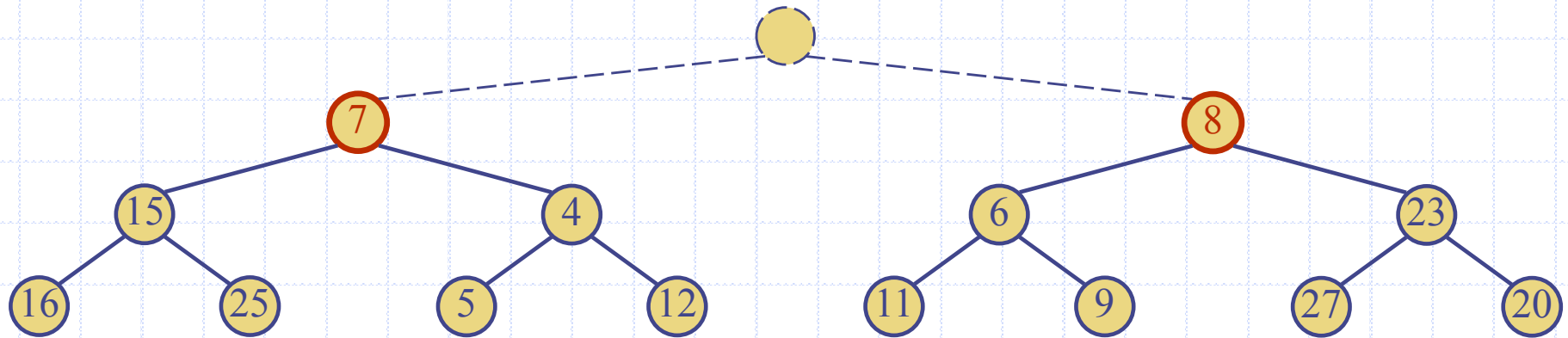


Example (contd.)

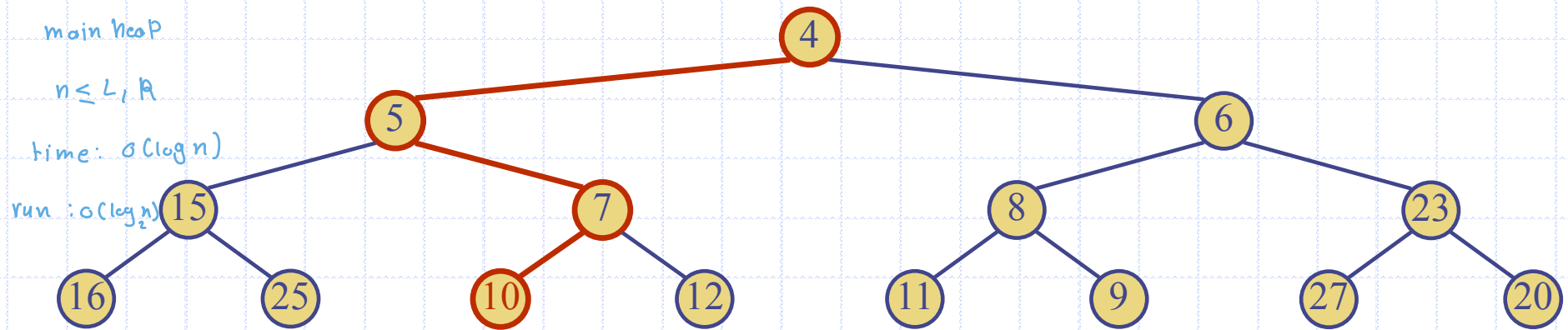
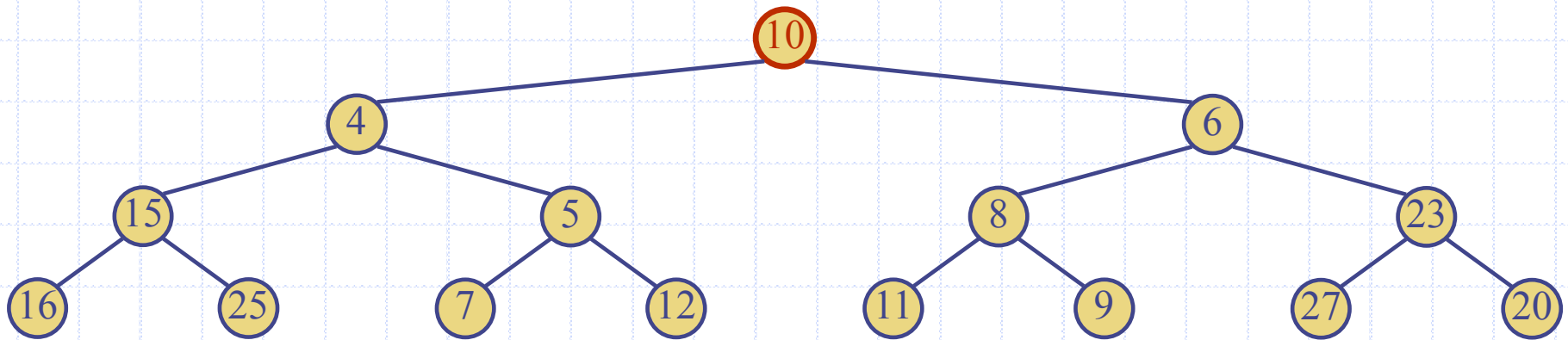
Insert level 2:

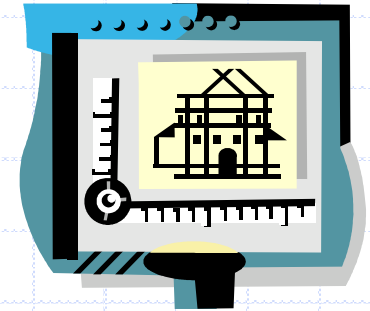


Example (contd.)



Example (end)





Analysis

- ◆ We visualize the worst-case time of a downheap with a proxy path that goes first right and then repeatedly goes left until the bottom of the heap (this path may differ from the actual downheap path)
- ◆ Since each node is traversed by at most two proxy paths, the total number of nodes of the proxy paths is $O(n)$
- ◆ Thus, bottom-up heap construction runs in $O(n)$ time
- ◆ Bottom-up heap construction is faster than n successive insertions and speeds up the first phase of heap-sort

